

SMART LIBRARY MANAGEMENT SYSTEM

Project Report

Course: Programming in Java

Project Title: Smart Library Management System

Project Type: Individual Academic Project

Student Name: Parth Yadav

Registration Number: 25BAI10400

Faculty / Guide: Muthusundari S

Institution: Vellore Institute of Technology Bhopal

Academic Year: 2026

1. Introduction

Libraries need to keep track of books, students and borrowing activities on a regular basis. When these activities are handled manually it can become difficult to maintain accurate information about which books are available and which books have been issued.

The Smart Library Management System is a simple console based application developed using Java. The main purpose of the project is to provide a basic system for managing books, students and borrowing activities in an organized way.

The application allows the user to add books, view books, search for books, add students, view students, issue books and return books. It also keeps a record of borrowing activities and calculates a fine when a book is returned after the allowed period.

This project was developed as an individual project for the Programming in Java course. It focuses on applying Java programming concepts such as classes, objects, constructors, methods, loops, conditional statements, ArrayList, Scanner and date handling.

The project is designed to be simple so that the main library operations can be clearly demonstrated without the need for a database or a graphical interface.

2. Problem Statement

Managing books and student borrowing manually can become difficult as the number of books and students increases. It can be challenging to know whether a particular book is available, who has borrowed it and whether it has been returned on time.

Manual management can also make it difficult to maintain borrowing history and calculate late return fines accurately.

The Smart Library Management System is developed to provide a simple computer based solution for these problems. The application manages book information, student information and borrowing records through a menu driven Java program.

The system checks the availability of books before issuing them and verifies the student information before completing an issue. When a book is returned the system calculates the number of days it was kept and calculates a fine if the return is late.

3. Functional Requirements

The functional requirements describe the main operations that the system must perform.

3.1 Book Management

The system allows the user to add new books.

Each book contains:

- Book ID
- Book Name
- Author Name
- Availability Status

The system prevents duplicate Book IDs.

The user can also view all books and their current availability status.

3.2 Student Management

The system allows the user to register students.

Each student contains:

- Student ID
- Student Name

The system prevents duplicate Student IDs.

The user can view all registered students.

3.3 Book Issue

The system allows a registered student to issue an available book.

Before issuing a book the system checks whether the Book ID exists and whether the Student ID exists.

The system also checks whether the selected book is already issued.

When the issue is successful the book status changes from Available to Issued.

The issue date is automatically recorded.

3.4 Book Return

The system allows the user to return an issued book.

The return date is automatically recorded using Java date handling.

The system calculates the number of days the book was kept.

A free period of 14 days is provided.

3.5 Fine Calculation

If a book is returned after the 14 day free period the system calculates a fine.

The fine is calculated at ₹10 for every late day.

The calculation used by the system is:

$\text{Fine} = \text{Late Days} \times ₹10$

3.6 Book Search

The system allows the user to search for a book using its Book ID.

The result displays the book details along with its current availability status.

3.7 Book History

The system maintains borrowing records during the current program session.

The history contains the Book ID, Student ID, issue date, return date, number of days kept and fine information.

The history also shows whether a book is currently issued or has already been returned.

3.8 Input Validation

The system checks for common input errors.

Empty book details are rejected.

Empty student details are rejected.

Duplicate Book IDs are rejected.

Duplicate Student IDs are rejected.

Invalid menu input is handled.

The system also checks whether a book and student exist before issuing a book.

4. Non Functional Requirements

4.1 Usability

The application uses a simple menu driven console interface. The user can select an operation by entering a number from the displayed menu.

The messages shown by the system are designed to make the result of each operation clear.

4.2 Reliability

The system performs checks before completing important operations such as issuing and returning books.

For example a book that is already issued cannot be issued again.

A book that is not currently issued cannot be returned.

4.3 Error Handling

The application handles common errors such as invalid menu input, empty details, duplicate IDs and invalid book or student information.

This prevents incorrect operations from being completed.

4.4 Maintainability

The application is divided into separate Java classes. Each class has a specific responsibility.

This makes the code easier to understand and makes future changes easier to implement.

4.5 Performance

The application is designed for a small library management scenario. Basic operations such as adding, searching and viewing records are performed quickly using Java ArrayList.

5. System Architecture

The Smart Library Management System follows a simple class based architecture.

The application is divided into five main Java classes.

Main.java

Main.java is the starting point of the application. It displays the menu and accepts input from the user.

Library.java

Library.java manages the main library operations. It handles books, students and borrowing records.

Book.java

Book.java represents a book in the library. It stores the Book ID, book name, author and availability status.

Student.java

Student.java represents a student. It stores the Student ID and student name.

BorrowRecord.java

BorrowRecord.java stores information related to a book borrowing activity. It stores the Book ID, Student ID, issue date and return date.

Architecture Flow

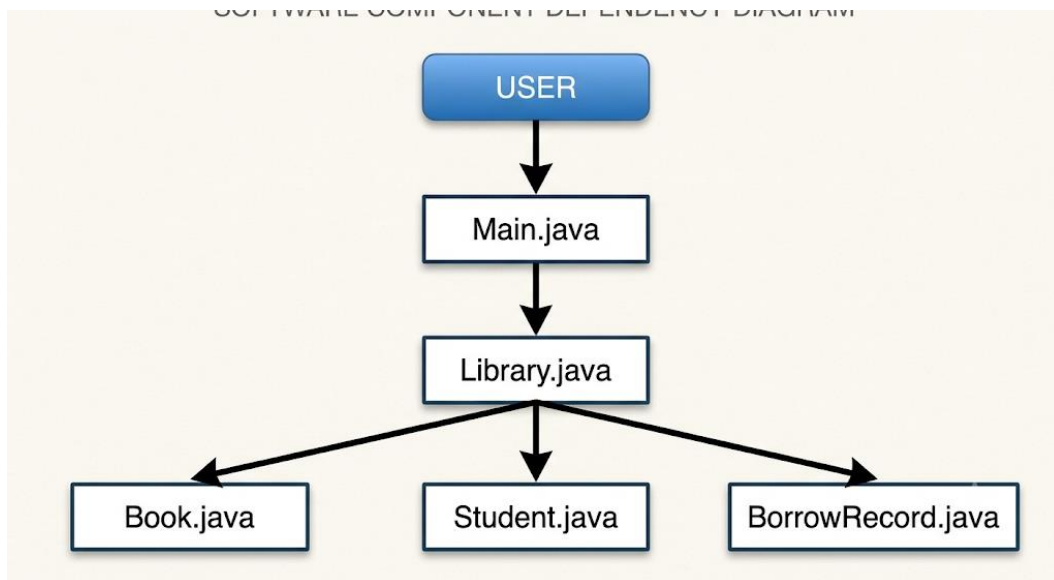
The basic flow of the application is:

User → Main.java → Library.java → Book / Student / BorrowRecord

The user interacts with the application through the console. Main.java receives the input and calls the appropriate method in Library.java.

Library.java performs the required operation using the Book, Student and BorrowRecord objects.

Diagram Placement



6. Design Diagrams

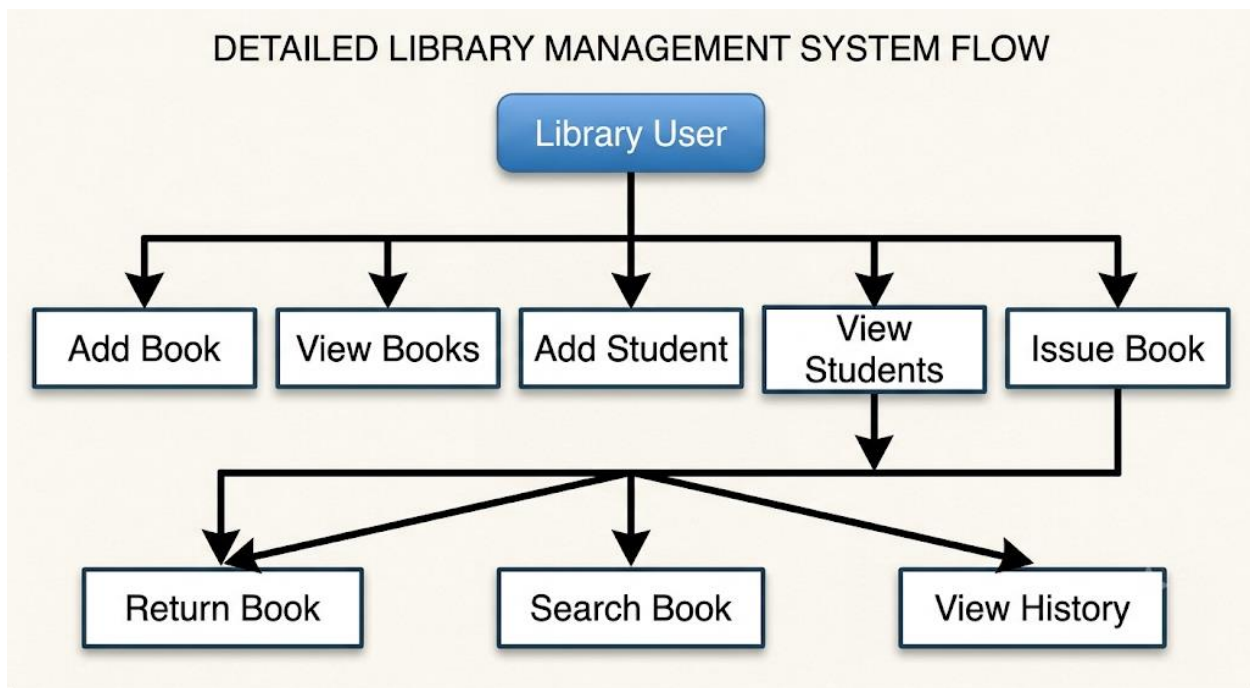
6.1 Use Case Diagram

The main actor in the system is the Library User.

The user can perform the following activities:

- Add Book
- View Books
- Add Student
- View Students
- Issue Book
- Return Book
- Search Book
- View Book History
- Exit System

Diagram Placement



6.2 Workflow Diagram

The workflow begins when the user starts the application.

The system displays the main menu.

The user selects an operation.

The system performs the selected operation and displays the result.

The menu is displayed again until the user chooses Exit.

General Workflow

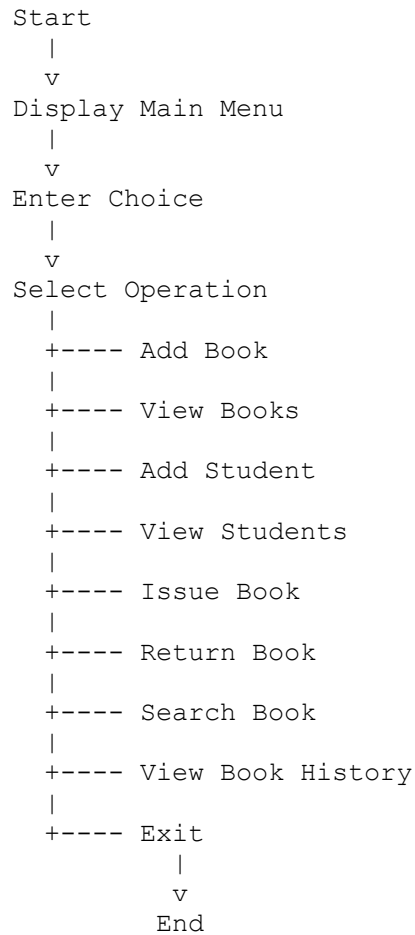
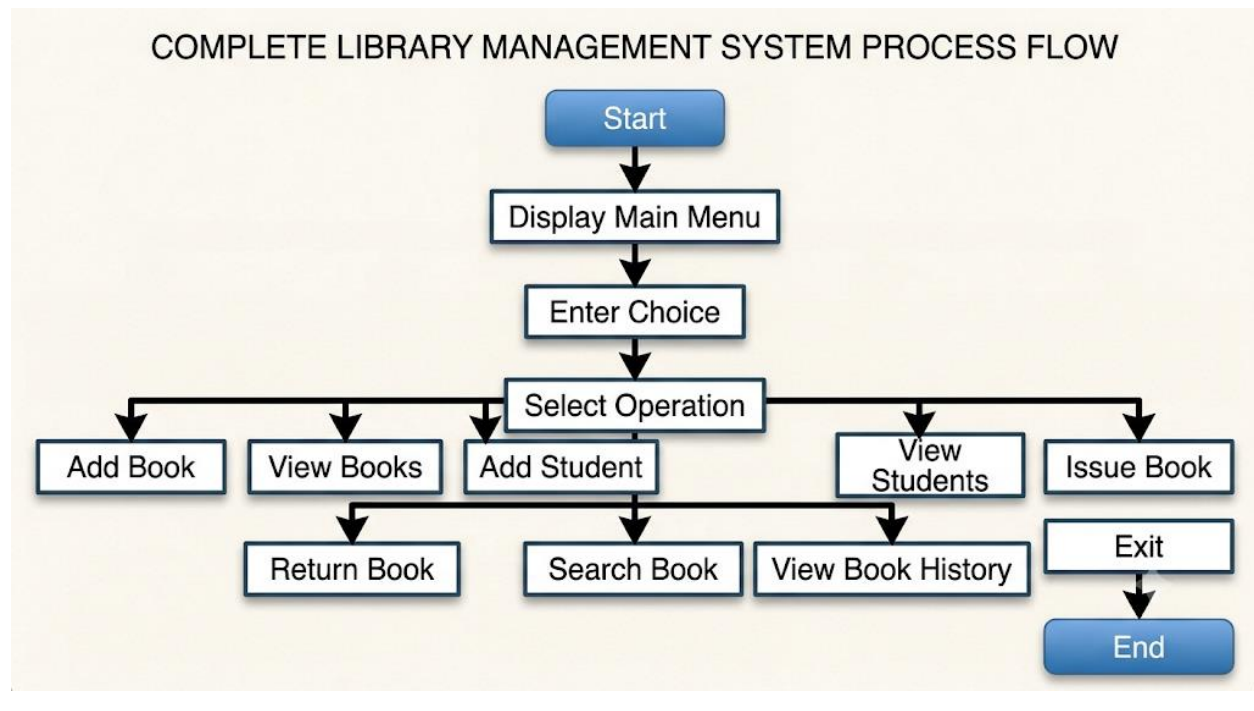


Diagram Placement



6.3 Sequence Diagram

The sequence diagram shows how the user interacts with the main classes during a book issue operation.

The user selects Issue Book from the menu.

Main.java receives the Book ID and Student ID.

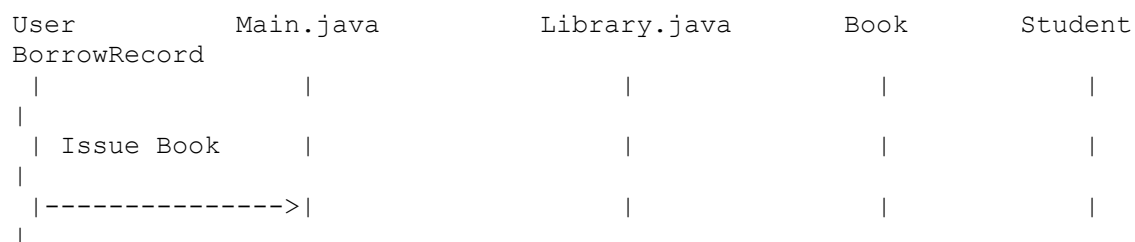
Main.java sends the information to Library.java.

Library.java checks whether the book and student exist.

It then checks whether the book is available.

If all conditions are satisfied the book is marked as issued and a BorrowRecord is created.

Sequence



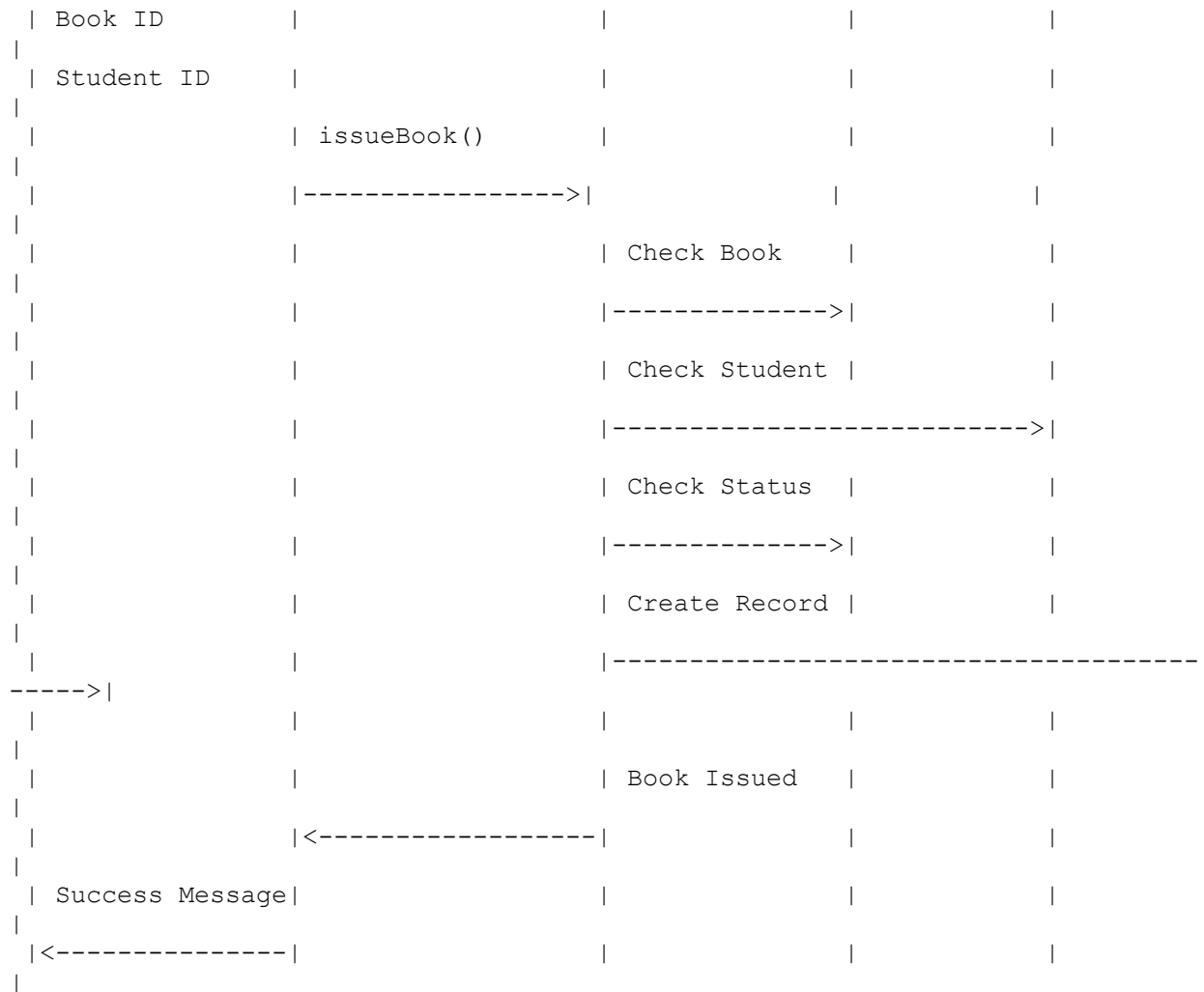
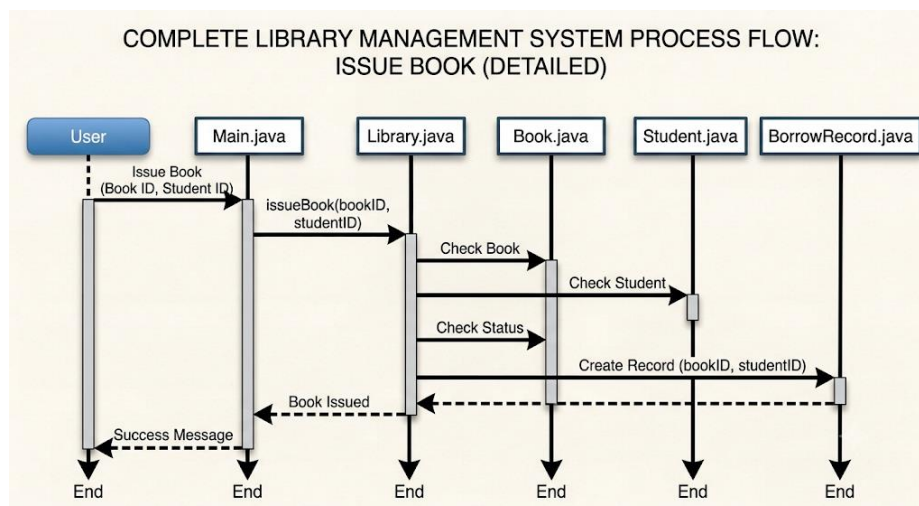


Diagram Placement



6.4 Class Diagram

The project contains five main Java classes.

Main

Main is responsible for starting the application, displaying the menu and accepting user input.

Library

Library manages the main operations of the application.

It contains:

- ArrayList of books
- ArrayList of students
- ArrayList of borrowing records
- Methods for adding and viewing books
- Methods for adding and viewing students
- Method for issuing books
- Method for returning books
- Method for searching books
- Method for viewing history

Book

Book contains:

- bookId
- bookName
- author
- available

Student

Student contains:

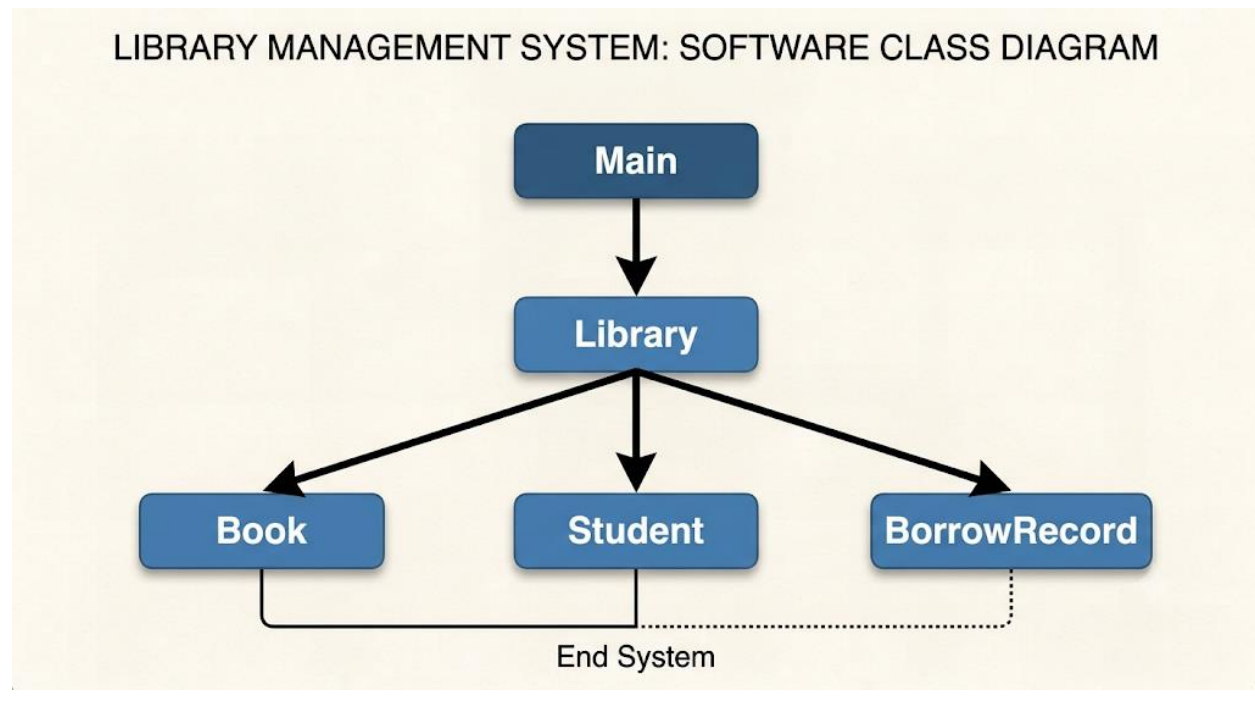
- studentId
- studentName

BorrowRecord

BorrowRecord contains:

- bookId
- studentId
- issueDate
- returnDate

Diagram Placement



6.5 ER Diagram

A database is not used in the current version of the project.

The application uses Java ArrayList for in memory storage.

Therefore an ER diagram is not required for the current implementation.

If required for documentation purposes the logical data relationship can be represented as:

```

BOOK
Book ID
Book Name
Author
Availability
  
```

```

|
| borrowed through
v
  
```

BORROW RECORD

Book ID

Student ID

Issue Date

Return Date

|
| belongs to
v

STUDENT

Student ID

Student Name

7. Design Decisions and Rationale

Several design decisions were made to keep the project simple and suitable for the Programming in Java course.

7.1 Console Based Application

A console interface was selected instead of a graphical interface.

The main reason was to focus on Java programming concepts and the core library management logic without adding unnecessary interface complexity.

7.2 Use of Separate Classes

The project uses separate classes for books, students, borrowing records and library operations.

This follows the basic principles of Object Oriented Programming and makes each part of the application easier to understand.

7.3 Use of ArrayList

ArrayList was selected for storing books, students and borrowing records.

It provides a simple way to store multiple objects while the program is running.

7.4 In Memory Storage

A database or permanent file storage was not added.

The project is intended to demonstrate the main Java concepts required by the course. Using in memory storage keeps the implementation simple and allows more focus on the actual programming logic.

The data is reset when the program is restarted.

7.5 Automatic Date Handling

LocalDate was used to record issue dates and return dates.

ChronoUnit.DAYS was used to calculate the number of days between the issue date and return date.

This avoids manually entering dates and allows the fine to be calculated automatically.

7.6 Validation

Input validation was included to prevent common mistakes.

The system checks for empty details, duplicate IDs and invalid operations before making changes to the library records.

8. Implementation Details

The application was implemented using Java.

The main classes used in the project are Main.java, Library.java, Book.java, Student.java and BorrowRecord.java.

8.1 Main Menu

The application starts with a menu containing nine options.

```
===== LIBRARY MANAGEMENT SYSTEM =====  
1. Add Book  
2. View Books  
3. Add Student  
4. View Students  
5. Issue Book  
6. Return Book  
7. Search Book  
8. View Book History  
9. Exit
```

The user selects an option and the corresponding operation is performed.

8.2 Adding Books

When the user selects Add Book the system asks for the Book ID, book name and author name.

The system checks whether any field is empty.

It also checks whether the Book ID already exists.

If the information is valid a new Book object is created and stored in the books ArrayList.

8.3 Adding Students

The user enters the Student ID and student name.

The system checks whether the information is empty and whether the Student ID already exists.

A new Student object is created when the information is valid.

8.4 Issuing Books

The user enters the Book ID and Student ID.

The system searches for the selected book and student.

If either one does not exist the operation is stopped.

The system then checks the availability of the book.

If the book is available its status is changed to Issued.

A BorrowRecord object is created and the current date is stored as the issue date.

8.5 Returning Books

The user enters the Book ID of the book being returned.

The system checks whether the book exists and whether it is currently issued.

The active BorrowRecord is then found.

The current date is stored as the return date.

The system calculates the number of days between the issue date and return date.

The book is marked as Available after the return.

8.6 Fine Calculation

The project provides a 14 day free period.

The number of late days is calculated using:

$$\text{Late Days} = \text{Days Kept} - 14$$

If the result is less than zero it is changed to zero.

The fine is calculated using:

$$\text{Fine} = \text{Late Days} \times ₹10$$

8.7 Search

The search function accepts a Book ID.

The system checks the books ArrayList and displays the details if the Book ID is found.

If no matching Book ID exists the system displays a book not found message.

8.8 Borrowing History

The history function displays every borrowing record stored during the current session.

For a returned book the system displays the return date, days kept, late days and fine.

For a book that is still issued the system displays its current issued status.

9. Screenshots and Results

The following screenshots can be added to show the working of the project.

9.1 Main Menu

```
===== LIBRARY MANAGEMENT SYSTEM =====
1. Add Book
2. View Books
3. Add Student
4. View Students
5. Issue Book
6. Return Book
7. Search Book
8. View Book History
9. Exit

Enter your choice: █
```

This screenshot shows the main menu of the Smart Library Management System.

9.2 Adding a Book

```
Enter your choice: 1
Enter Book ID: A001
Enter Book Name: Java
Enter Author Name: Babu
Book added successfully.
```

This screenshot shows a new book being added successfully.

9.3 Viewing Books

```
Enter your choice: 2

===== ALL BOOKS =====
Book ID: A001
Book Name: Java
Author: Babu
Status: Available
-----
```

This screenshot shows the list of books and their availability status.

9.4 Adding a Student

```
Enter your choice: 3
Enter Student ID: 25BAI10400
Enter Student Name: Parth
Student added successfully.
```

This screenshot shows a student being registered in the system.

9.5 Issuing a Book

```
Enter your choice: 5
Enter Book ID: A001
Enter Student ID: 25BAI10400
Book issued successfully.
Issue Date: 2026-09-14
Return within 14 days to avoid a fine.
```

This screenshot shows a book being issued to a registered student.

9.6 Searching for a Book

```
Enter your choice: 7
Enter Book ID to search: A001

===== BOOK FOUND =====
Book ID: A001
Book Name: Java
Author: Babu
Status: Available
-----
```

This screenshot shows the search operation using a Book ID.

9.7 Returning a Book

```
Enter your choice: 6
Enter Book ID: A001

===== BOOK RETURN =====
Book ID: A001
Issue Date: 2026-09-14
Return Date: 2026-09-14
Days Kept: 0
Late Days: 0
Fine: Rs. 0
Book returned successfully.
```

This screenshot shows the return operation and automatic fine calculation.

9.8 Book History

```
Enter your choice: 8

===== BOOK HISTORY =====
Book ID: A001
Student ID: 25BAI10400
Issue Date: 2026-09-14
Return Date: 2026-09-14
Days Kept: 0
Late Days: 0
Fine: Rs. 0
Status: Returned
-----
```

This screenshot shows the borrowing history containing issue information, return information and fine details.

9.9 Result

The project was tested through the console using different library operations.

The main operations including adding books, adding students, issuing books, searching books, returning books and viewing borrowing history worked as expected.

The system also handled duplicate IDs, invalid input and incorrect issue or return operations.

10. Testing Approach

Testing was performed manually through the console.

Each major feature was tested using normal input as well as incorrect input.

Test Case 1: Add Book

Input: Valid Book ID, book name and author

Expected Result: Book should be added successfully.

Result: Passed

Test Case 2: Duplicate Book

Input: Book ID that already exists

Expected Result: System should reject the duplicate Book ID.

Result: Passed

Test Case 3: Add Student

Input: Valid Student ID and student name

Expected Result: Student should be added successfully.

Result: Passed

Test Case 4: Duplicate Student

Input: Student ID that already exists

Expected Result: System should reject the duplicate Student ID.

Result: Passed

Test Case 5: Issue Book

Input: Existing Book ID and existing Student ID for an available book

Expected Result: Book should be issued successfully.

Result: Passed

Test Case 6: Issue Already Issued Book

Input: Book ID of a book that is already issued

Expected Result: System should prevent the issue.

Result: Passed

Test Case 7: Search Book

Input: Existing Book ID

Expected Result: Book information should be displayed.

Result: Passed

Test Case 8: Return Book

Input: Book ID of an issued book

Expected Result: Book should be returned and the return information should be displayed.

Result: Passed

Test Case 9: Return Available Book

Input: Book ID of a book that is not currently issued

Expected Result: System should prevent the return operation.

Result: Passed

Test Case 10: View History

Input: Select View Book History

Expected Result: Borrowing records should be displayed.

Result: Passed

Test Case 11: Invalid Menu Input

Input: Non numeric menu input or an invalid menu number

Expected Result: System should display an error message.

Result: Passed

Test Case 12: Empty Input

Input: Empty Book ID, book name, author or Student ID

Expected Result: System should reject the empty information.

Result: Passed

11. Challenges Faced

One of the main challenges during development was organizing the different library operations into separate classes.

The project contains different types of information such as books, students and borrowing records. Keeping these responsibilities separate helped make the program easier to manage.

Another challenge was handling book availability. When a book is issued its status must change to Issued. When the same book is returned its status must change back to Available.

Another challenge was implementing the fine calculation. The program needs to calculate the number of days between the issue date and return date. Java LocalDate and ChronoUnit were used to handle this calculation.

Input validation was also an important part of the project. The program needed to handle invalid menu input, duplicate IDs and incorrect book or student information without terminating unexpectedly.

Testing the complete workflow also required checking different situations such as issuing an already issued book and trying to return a book that is already available.

12. Learnings and Key Takeaways

This project provided practical experience with Java programming and Object Oriented Programming.

One important learning was how classes and objects can be used to represent real world entities. The Book, Student and BorrowRecord classes represent different parts of a library system.

The project also helped in understanding how ArrayList can be used to store and manage multiple objects.

Another important learning was using methods to divide a larger program into smaller operations. Each library operation is handled by a separate method which makes the program easier to understand.

The project also provided experience with user input through Scanner and input validation.

Working with LocalDate provided an understanding of Java date and time handling.

The project also showed the importance of testing. Testing different valid and invalid situations helped identify problems and confirm that the application behaved correctly.

Overall the project helped connect the Java concepts learned in class with a practical application.

13. Future Enhancements

The current version provides the basic features required for a simple library management system. Several improvements could be added in the future.

Permanent Data Storage

A database or file based storage system could be added so that books, students and borrowing records remain available after the application is closed.

Login System

A login system could be added to restrict access to library management operations.

Book Categories

Books could be organized into categories such as programming, science, mathematics and literature.

Borrowing Limits

The system could allow a maximum number of books for each student.

Graphical User Interface

A graphical interface could be added to make the system easier to use.

Improved Search

The search feature could be extended to search using book name or author name in addition to Book ID.

Reports

The system could generate reports showing issued books, returned books, overdue books and collected fines.

14. References

The following resources were used for understanding Java concepts during the development of the project.

1. Java Documentation for learning Java classes, methods and standard library features.
 2. Java documentation for ArrayList and collection handling.
 3. Java documentation for Scanner and console input.
 4. Java documentation for LocalDate and date handling.
 5. Course materials and classroom learning from the Programming in Java course.
 6. GitHub documentation for repository management and project documentation.
-

Conclusion

The Smart Library Management System is a simple Java based console application developed to manage basic library activities.

The system provides features for managing books and students along with issuing and returning books. It also provides book search, borrowing history, availability checking and automatic fine calculation.

The project demonstrates important Java concepts such as classes, objects, constructors, methods, loops, conditional statements, ArrayList, Scanner and date handling.

The project was designed with simplicity in mind so that the core programming concepts and library management workflow could be clearly demonstrated.

Through this project I gained practical experience in designing a small Java application, organizing code into multiple classes, validating user input and testing different program operations.

The current system can also be extended in the future with permanent storage, login functionality and a graphical interface.